

Chapter 9

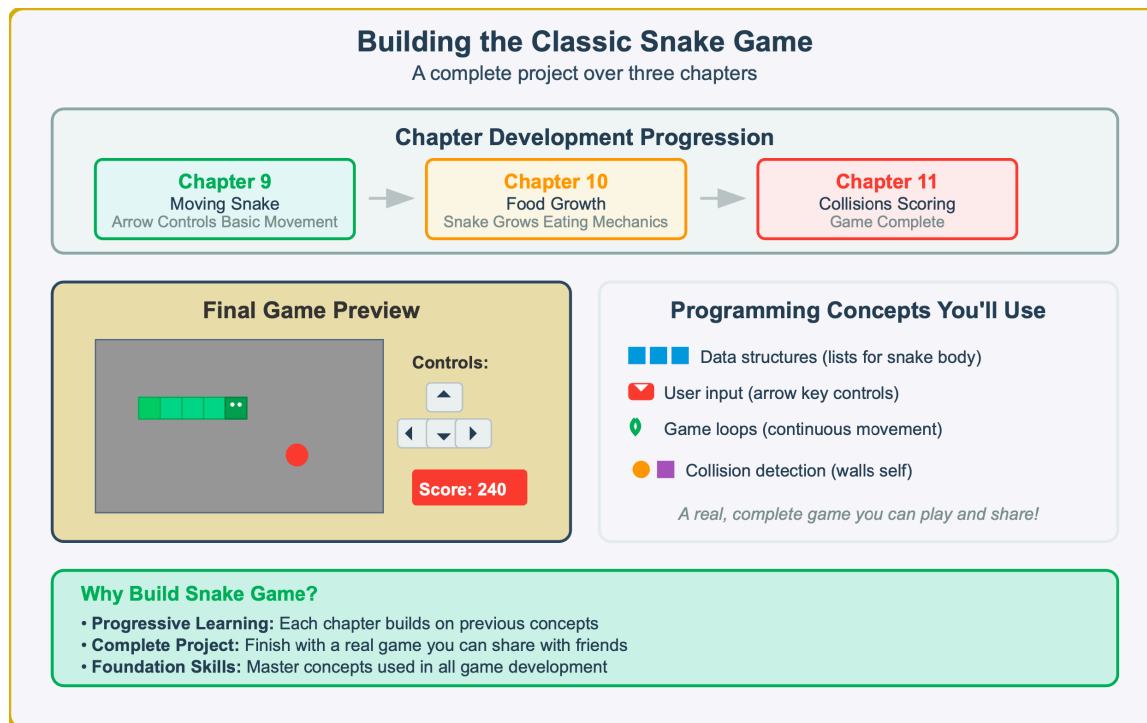


Figure 9.1 — Building the Classic Snake Game — A Complete Project Over Three Chapters. This overview shows the progressive development of a complete Snake game across three chapters, from basic movement to food mechanics to collision detection and scoring.

Creating the Screen

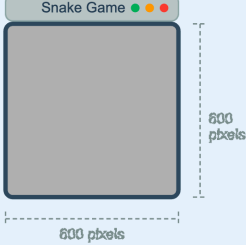
Setting up a proper game screen

Setup Code

```
import turtle
import time

# Set up the screen
screen = turtle.Screen()
screen.title("Snake Game")
screen.bgcolor("black")
screen.setup(width=600, height=600)
screen.tracer(0)
# Turn off animation
print("Snake Game Board Created!")
screen.exitonclick()
```

Screen Setup Result



Key Features

- Black background
- 600×600 square
- Animation off
- Manual control

Perfect foundation for our Snake game!

Console Output Setup Results

```
Snake Game Board Created!
Screen size: 600×600 pixels
```

Setup Explanation:

- `screen.tracer(0)`: Disables automatic animation for manual control
- **600×600 size**: Creates perfect square game board for Snake movement

This screen setup provides the foundation for all Snake game development!

Figure 9.2 — Creating the Snake Game Screen — Setting Up a Proper Game Screen. This code creates a 600×600 black game window with animation disabled for manual control, providing the perfect foundation for Snake game development.

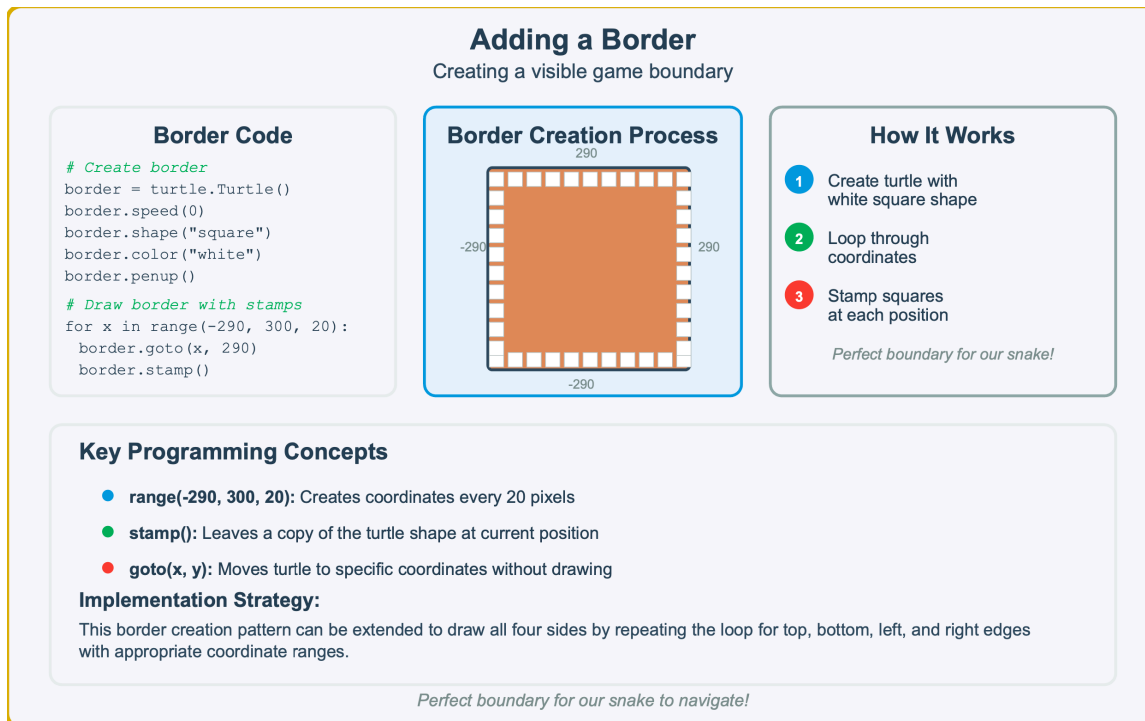
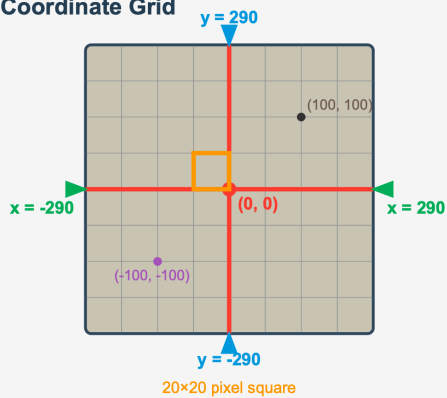


Figure 9.3 — Adding a Border — Creating a Visible Game Boundary. This code creates a white square border around the game area using turtle stamps positioned every 20 pixels, providing clear visual boundaries for Snake movement.

Understanding Coordinates

How our 600×600 screen coordinate system works

Coordinate Grid



Coordinate System Rules

- Center of screen is (0, 0)
- Y-axis: +290 (top) to -290 (bottom)
- X-axis: -290 (left) to +290 (right)
- Each grid square is 20×20 pixels
- Snake moves in 20-pixel steps

Corner Examples:

- Top-left corner: (-290, 290)
- Top-right corner: (290, 290)
- Bottom-left corner: (-290, -290)
- Bottom-right corner: (290, -290)

Key Programming Insight

This coordinate system allows precise Snake movement: each direction change moves exactly 20 pixels, creating the classic grid-based gameplay. Understanding these coordinates is essential for collision detection and food placement.

Master these coordinates and you master Snake movement!

Figure 9.4 — Understanding Coordinates — How Our 600×600 Screen Coordinate System Works. This coordinate grid shows the game screen layout with center at (0,0), boundaries at ±290, and 20×20 pixel squares for precise Snake movement.

Creating the Snake Head

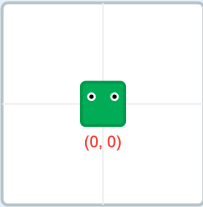
Setting up our snake player

Snake Head Code

```
# Create the snake head
snake = turtle.Turtle()
snake.speed(0)
snake.shape("square")
snake.color("green")
snake.penup()
snake.goto(0, 0)
print("Snake head created at center!")
```

Ready to move! Next we'll add controls.

Snake Head Result



Green square at game center

Setup Steps

- 1 Create new turtle object
- 2 Set speed to fastest (0)
- Make it square shaped
- Color it green
- 5 Position at center (0,0)

Snake head setup complete!

Key Methods Explained

- **speed(0):** Sets fastest animation speed for smooth movement
- **shape("square"):** Creates a 20×20 pixel square shape
- **penup():** Prevents drawing trails when moving
- **goto(0, 0):** Moves turtle to center of screen

Why These Settings?

- Square shape fits perfectly in our 20×20 pixel grid system
- Green color makes the snake easily visible against the black background
- Center position (0,0) provides equal movement space in all directions

Figure 9.5 — Creating the Snake Head — Setting Up Our Snake Player. This code creates a green square turtle positioned at the center of the screen, serving as the player—controlled Snake head with optimal speed settings and no drawing trail.

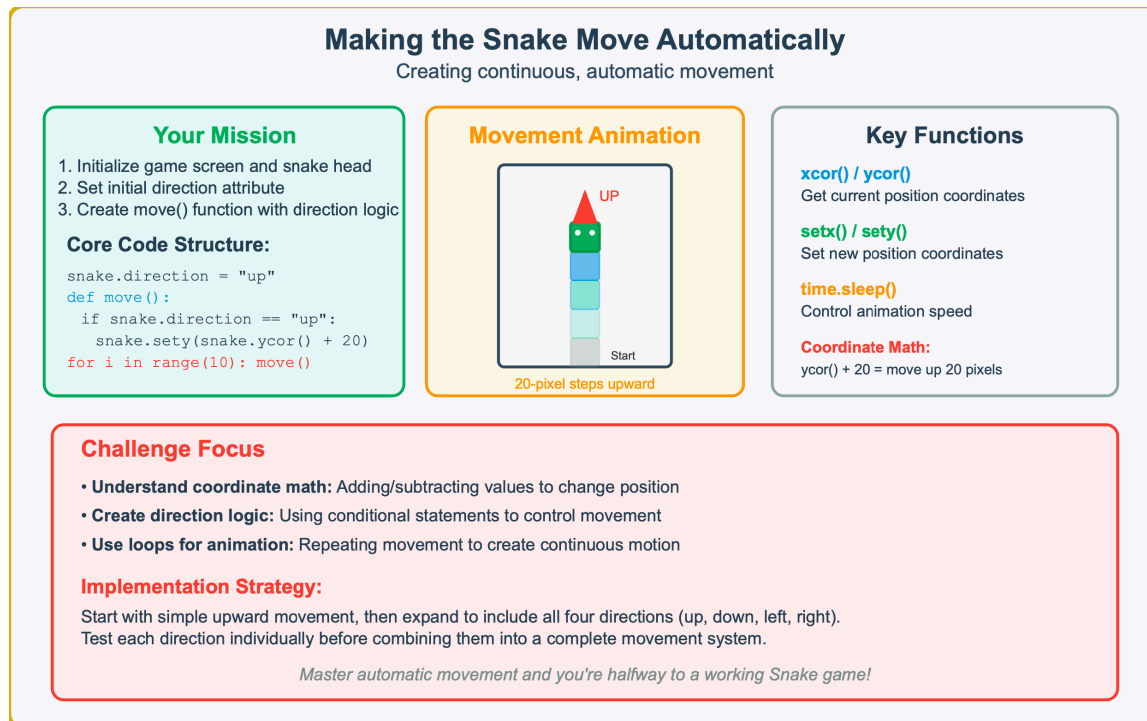


Figure 9.6 — Making the Snake Move Automatically — Creating Continuous, Automatic Movement. This implementation shows how to create direction attributes, movement functions using coordinate math, and animation loops to make the Snake move continuously upward in 20—pixel steps.

Adding Keyboard Controls

Give the player control over the snake!

Your Mission

1. Set initial direction to "stop"
2. Create direction functions
3. Bind arrow keys to functions
4. Create continuous game loop

Direction Functions:

```
snake.direction = "stop"
def go_up():
    snake.direction = "up"
def go_down():
    snake.direction = "down"
```

Key Binding Setup

```
screen.listen()
screen.onkey(go_up, "Up")
screen.onkey(go_down, "Down")
screen.onkey(go_left, "Left")
screen.onkey(go_right, "Right")

def go_left():
    snake.direction = "left"
def go_right():
    snake.direction = "right"
```

Arrow Key Controls



Real-Time Control

Press any arrow key to change
snake direction instantly

How Keyboard Controls Work

Event-Driven Programming Process:

1. **screen.listen()**: Tells the screen to start listening for keyboard events
2. **screen.onkey()**: Binds specific keys to specific functions
3. **Direction functions**: Change the snake.direction attribute when called
4. **Game loop**: Continuously checks direction attribute and moves accordingly

Key Programming Insight:

The direction functions don't move the snake directly - they only change the direction attribute. The game loop reads this attribute and handles the actual movement.

Now the player has full control over snake movement!

Figure 9.7 — Adding Keyboard Controls — Give the Player Control Over the Snake! This implementation shows how to create direction functions, bind arrow keys to those functions, and enable real—time player control over Snake movement.

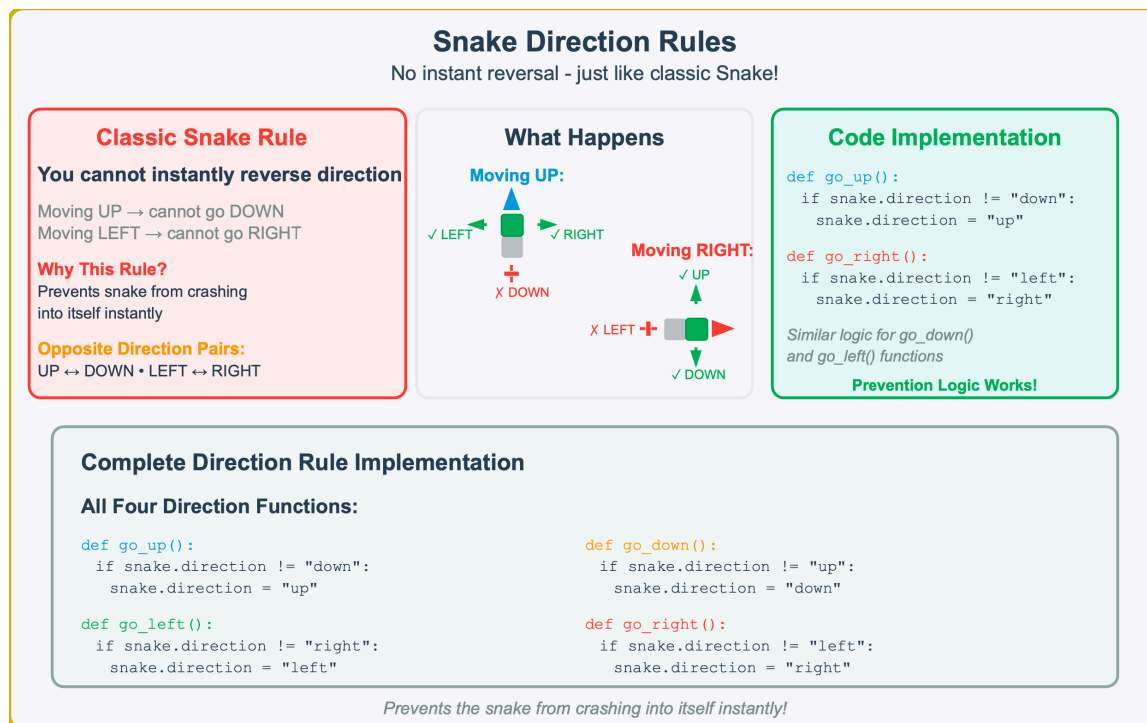


Figure 9.8 — Snake Direction Rules — No Instant Reversal — Just Like Classic Snake!
This implementation shows how to prevent the snake from instantly reversing direction by checking current direction before allowing new direction changes, maintaining classic Snake gameplay mechanics.

Direction Control Challenge

Add realistic Snake direction rules

Your Mission:

1. Start with your existing keyboard control code
2. Modify the four direction functions with conditional checks

What Stays The Same:

- ✓ Game setup • ✓ Snake object • ✓ move() function
- ✓ Keyboard bindings • ✓ Main game loop (while True:)

Function Updates Required:

BEFORE:

```
def go_up():  
    snake.direction = "up"
```

➔

AFTER:

```
def go_up():  
    if snake.direction != "down":  
        snake.direction = "up"
```

Complete Function Pattern:

```
def go_up(): if snake.direction != "down": snake.direction = "up"  
def go_down(): if snake.direction != "up": snake.direction = "down"  
def go_left(): if snake.direction != "right": snake.direction = "left"
```

Focus: Only modify the four direction functions!

Small change, big impact on gameplay realism!

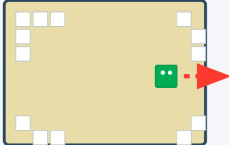
Figure 9.9 — Snake Direction Control Challenge — Preventing Impossible Moves. This exercise demonstrates how to add conditional logic to prevent the snake from reversing into itself, making the game more realistic by blocking opposite direction changes.

Snake Border Collision

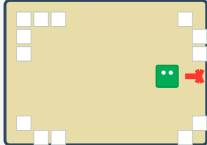
Make the snake stop when it hits the border

Grid-Based Game Challenge:
Define boundaries for your game world — Stop the snake from disappearing into the void!

Before vs After:

BEFORE:

Snake disappears!

AFTER:

Snake stops safely!

Game Boundaries to Check:

- Top edge: $y = 290$
- Right edge: $x = 290$
- Bottom edge: $y = -290$
- Left edge: $x = -290$

Goal:
Detect when snake hits boundary and stop movement

Essential for creating proper game boundaries!

Figure 9.10 — Snake Border Collision — Boundary Detection Challenge. This visual comparison shows the importance of implementing boundary detection to prevent the snake from disappearing off—screen, demonstrating proper game world limits.

Boundary Check Implementation

Add collision detection to your snake game

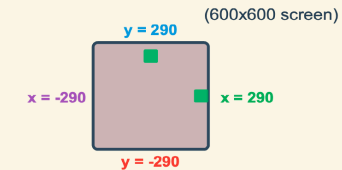
Your Mission:

1. Create `check_boundaries()` function • 2. Add boundary collision logic • 3. Integrate into main game loop • 4. Keep all existing code intact

check_boundaries() Function:

```
def check_boundaries():  
    # Horizontal Check  
    if snake.xcor() > 290 or snake.xcor() < -290:  
        snake.direction = "stop"  
        print("Hit side wall! Snake stopped.")  
  
    # Vertical Check  
    if snake.ycor() > 290 or snake.ycor() < -290:  
        snake.direction = "stop"
```

Boundary Coordinates:



Collision Logic:

`xcor() > 290` → Right wall hit
`xcor() < -290` → Left wall hit

Game Loop Integration:

```
while True: ... move() ... check_boundaries() ...  
Simply add the boundary check after each move in your main game loop
```

Simple addition, major improvement to game behavior!

Figure 9.11 — Boundary Check Implementation — Adding Collision Detection. This implementation guide shows the complete `check_boundaries()` function with coordinate logic and integration steps for preventing the snake from moving beyond game boundaries.

Snake Wrap-Around Alternative

Make the snake teleport to the opposite side!

Alternative Approach:

Instead of stopping, make the snake wrap around to the opposite side!

Wrap-Around Effect:



Coordinate Mapping:

Horizontal Wrapping:

$x > 290 \rightarrow x = -290$

$x < -290 \rightarrow x = 290$

Vertical Wrapping:

$y > 290 \rightarrow y = -290$

$y < -290 \rightarrow y = 290$

wrap_around() Function:

```
def wrap_around():
    if snake.xcor() > 290: snake.setx(-290)    if snake.ycor() > 290: snake.sety(-290)
    elif snake.xcor() < -290: snake.setx(290)  elif snake.ycor() < -290: snake.sety(290)
```

Game Loop Update:

```
wrap_around()
# Use instead of
# check_boundaries()
```

Creates a "Pac-Man" style world with no walls!

Figure 9.12 — Snake Wrap—Around Alternative — Teleportation Method. This alternative to boundary collision shows how to implement wrap—around teleportation where the snake appears on the opposite side when reaching screen edges, creating a seamless “Pac—Man” style game world.

Setting Up Snake Body Structure

From single head to complete snake with body segments

Your Mission:

- 1. Create snake head
- 2. Initialize segments list
- 3. Create body segments
- 4. Position segments behind head
- 5. Verify setup with print statements

Snake Structure:

(0, 0)  **HEAD**

(0, -20)  Segment 0

(0, -40)  Segment 1

(0, -60)  Segment 2

List Structure:

`segments = [ ]`

`segments[0]` → Body segment

`segments[1]` → Body segment

`segments[2]` → Body segment

...

Each element is a turtle object

Key Code Structure:

```
segments = []  
  
for i in range(3):  
    new_segment = turtle.Turtle()  
    segments.append(new_segment)
```

Critical Steps:

```
new_segment.goto(0, -20 * (i + 1))  
# Position behind head  
segments.append(new_segment)  
# Add to list
```

Foundation for a growing, following snake body!

Figure 9.13 — Setting Up Snake Body Structure — Segments and Lists. This implementation guide shows how to create a multi—segment snake using a list structure, with proper positioning of body segments behind the head and the code foundation for snake growth mechanics.

Snake Body Following Logic

Making body segments elegantly follow the head

Your Mission:

- 1. Start with horizontal body structure
- 2. Refine move() function
- 3. Implement chain-reaction movement
- 4. Demonstrate with animation

Chain-Reaction Movement:

BEFORE:

AFTER:

move() Function Logic (in order):

- 1 **Move Trailing Segments:**

```
for i in range(len(segments)-1, 0, -1):
    segments[i].goto(segments[i-1].xcor(), segments[i-1].ycor())
```
- 2 **Move First Segment:**

```
if len(segments) > 0: segments[0].goto(head.xcor(), head.ycor())
```
- 3 **Move Head:**

```
if head.direction == "right": head.setx(head.xcor() + 20)
```

Demo Animation:

```
for i in range(10):
    move()
    screen.update()
    time.sleep(0.5)
```

Each segment follows the one in front - like a real snake!

Figure 9.14 — Snake Body Following Logic — Chain—Reaction Movement. This implementation demonstrates the three—step movement algorithm that creates realistic snake behavior, where each body segment follows the position of the segment in front of it, creating smooth chain—reaction movement.

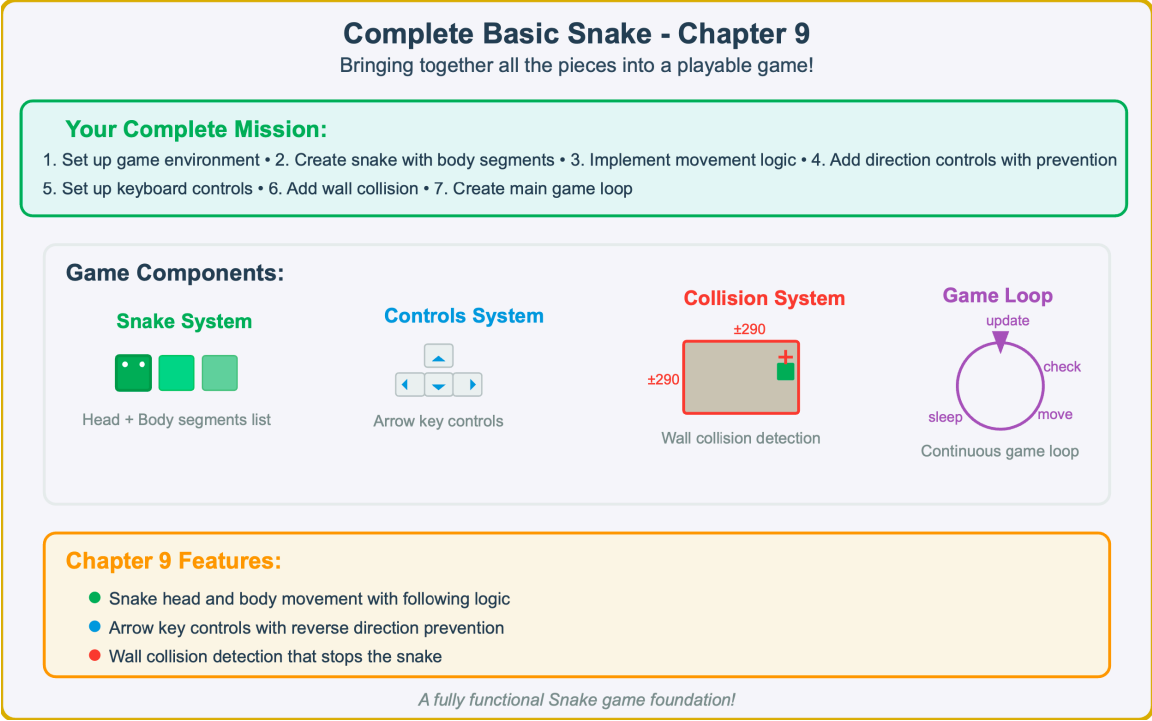


Figure 9.15 — Complete Basic Snake — Chapter 9 Overview. This comprehensive summary shows all the key components that come together to create a fully functional Snake game: the snake system with following logic, arrow key controls with direction prevention, collision detection, and the continuous game loop.

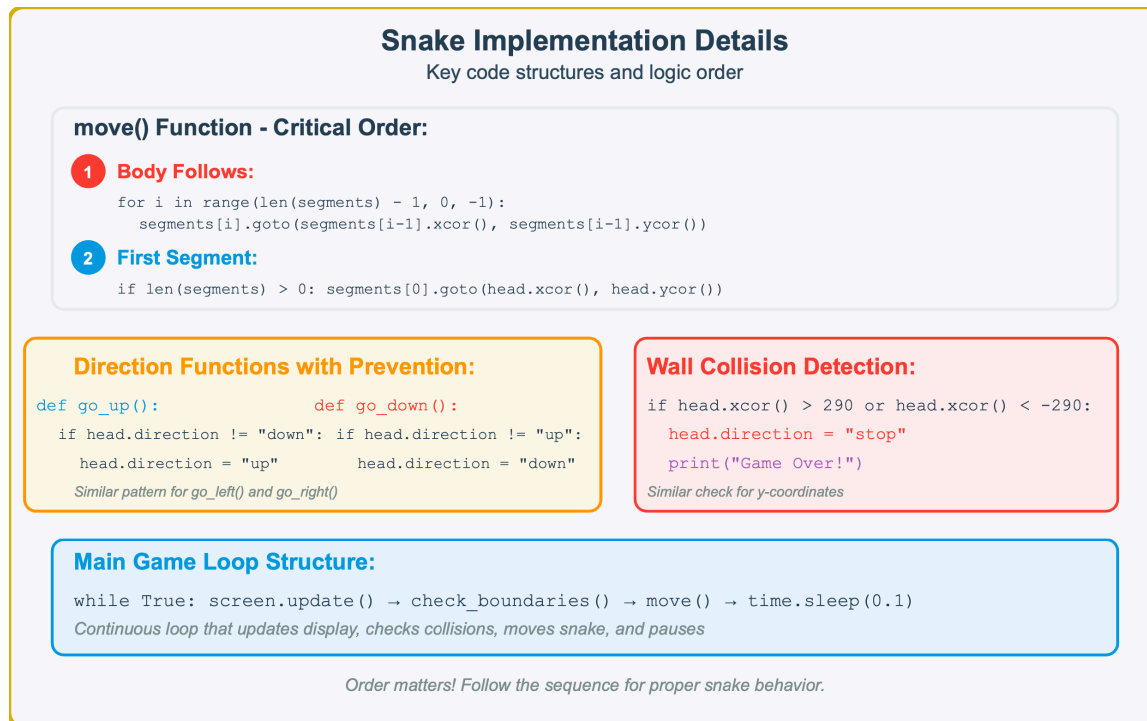


Figure 9.16 — Snake Implementation Details — Code Structure and Logic Order. This technical reference shows the critical implementation details for snake movement, direction control with prevention logic, collision detection, and the main game loop sequence that must be followed for proper snake behavior.

Complete Snake Setup Structure

Game environment and initialization code

Game Environment Setup:

```
import turtle, time
screen = turtle.Screen()
screen.title("Snake Game - Chapter 9"), screen.bgcolor("black"), screen.setup(600, 600)
```

Snake Creation:

```
# Create head
head = turtle.Turtle()
head.shape("square")
head.color("green")
head.direction = "stop"
```

Keyboard Controls Setup:

```
screen.listen()
screen.onkey(go_up, "Up")
screen.onkey(go_down, "Down")
screen.onkey(go_left, "Left")
screen.onkey(go_right, "Right")
```

Player Instructions:

```
print("=== Snake Game - Chapter 9 ===")
print("Use arrow keys to control the snake")
print("Avoid hitting the walls!")
```

Ready to play! All Chapter 9 concepts combined into one game.

Figure 9.17 — Complete Snake Setup Structure — Game Initialization Code. This setup reference shows the complete initialization sequence for the Snake game, including environment configuration, snake object creation, keyboard control binding, and player instruction display.

Try It Yourself: Snake Customization

Create a program that lets users customize their snake

Starting Code Structure:

```
import turtle
# Your code here:
# 1. Ask user for snake color
# 2. Ask user for starting direction
# 3. Ask user for starting speed
# 4. Create customized snake
screen = turtle.Screen()
```

User Input Examples:

- Snake Color: "red", "blue", "purple", "orange"
- Starting Direction: "up", "down", "left", "right"
- Starting Speed: "1" (slow), "5" (medium), "10" (fast)

💡 Hint: Use input() function to get user responses

Customization Results:



Red



Blue



Purple



Orange



Direction
Control



Speed Options

Interactive Features to Implement:

- Color selection from user input
- Direction preference setting
- Speed control options
- Visual confirmation of choices

Challenge: Add validation to ensure users enter valid options!

Make it interactive! Let users create their perfect snake.

Figure 9.18 — Try It Yourself — Snake Customization Challenge. This programming exercise challenges students to create an interactive program that lets users customize their snake's appearance and behavior through input prompts for color, starting direction, and speed settings.

Try It Yourself: Multiple Snakes

Create two snakes controlled by different keys

Starting Code Structure:

```
import turtle, time
screen = turtle.Screen(), screen.setup(800, 600), screen.tracer(0)
# Your code here:
# 1. Create snake1 (WASD) • # 2. Create snake2 (arrows)
```

Two-Player Game:



Controls:

Player 1 (WASD):



Player 2 (Arrows):



Challenge Requirements:

- Two separate snake objects with different colors
- Simultaneous movement in main game loop
- Independent movement functions for each snake
- Key bindings for both control schemes

Create competitive multiplayer Snake gameplay!

Figure 9.19 — Try It Yourself — Multiple Snakes Challenge. This advanced programming exercise challenges students to create a two—player Snake game with independent snakes controlled by different key schemes (WASD vs arrow keys), requiring separate snake objects, movement functions, and simultaneous gameplay.

Try It Yourself: Snake Maze

Create a simple maze for your snake to navigate

Starting Code Structure:

```
import turtle
screen = turtle.Screen(), screen.setup(600, 600), screen.bgcolor("black")
# Your code here:
# 1. Create walls in the middle of the screen
```

Maze Layout Example:

Maze Elements:

- Border walls
- Maze walls
- Snake
- Navigation path

Key Programming Tasks:

- Create wall objects and position strategically
- Check collision between snake and walls
- Stop movement when collision detected

Implementation Challenges:

- Create wall objects and position them strategically
- Check collision between snake head and wall objects
- Stop snake movement when wall collision detected
- Design interesting maze layout with multiple paths

Add strategy and challenge to your Snake game!

Figure 9.20 — Try It Yourself — Snake Maze Challenge. This advanced programming exercise challenges students to create a maze—based Snake game by adding wall objects, implementing collision detection between the snake and maze walls, and designing strategic obstacle layouts that require careful navigation.